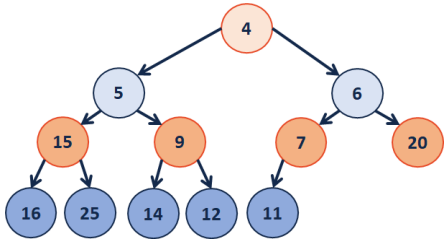
CS 225

#28: Heaps & Heap Operations

April 22-24, 2026 V. Kindratenko

Implementing a (min)Heap as an Array



4	5	6	15	9	7	20	16	25	14	12	11			

leftChild(index):

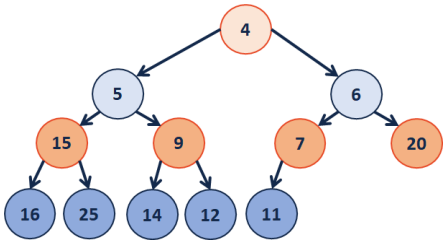
rightChild(index):

parent(index):

A complete binary tree T is a min-heap if:

-
-

Heap Operation: insert / heapifyUp:



-	4	5	6	15	9	7	20	16	25	14	12	11			

Heap.cpp (partial)

```
1  template <class T>
2  void Heap<T>::_insert(const T & key) {
3    // Check to ensure there's space to insert an element
4    // ...if not, grow the array
5    if ( size_ == capacity_ ) { _growArray(); }
6
7    // Insert the new element at the end of the array
8    item_[++size] = key;
9
10   // Restore the heap property
11   _heapifyUp(size);
12 }
```

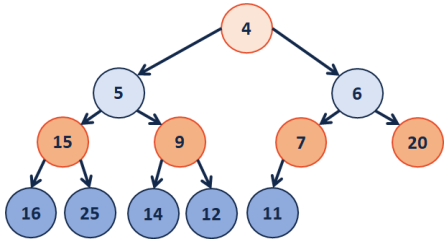
```
1  template <class T>
2  void Heap<T>::_heapifyUp( ____________________ ) {
3    if ( index > _________ ) {
4      if ( item_[index] < item_[ parent(index) ] ) {
5         std::swap( item_[index], item_[ parent(index) ] );
6         _heapifyUp( ________________ );
7      }
8    }
9  }
```

insert Running Time?

removeMin Running Time?

Heap Operation: removeMin / heapifyDown:

removeMin:



-	4	5	6	15	9	7	20	16	25	14	12	11			

Heap.cpp (partial)

```
1  template <class T>
2  void Heap<T>::_removeMin() {
3    // Swap with the last value
4    T minValue = item_[1];
5    item_[1] = item_[size_];
6    size--;
7
8    // Restore the heap property
9    heapifyDown();
10
11   // Return the minimum value
12   return minValue;
13 }
```

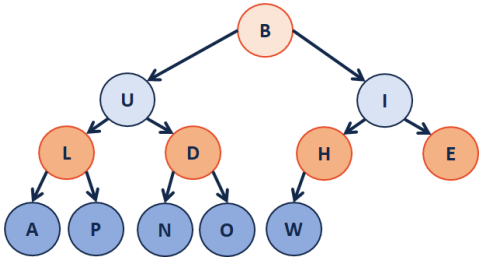
```
1  template <class T>
2  void Heap<T>::_heapifyDown(int index) {
3    if ( !_isLeaf(index) ) {
4      T minChildIndex = _minChild(index);
5      if ( item_[index] ___ item_[minChildIndex] ) {
6         std::swap( item_[index], item_[minChildIndex] );
7         _heapifyDown( _________________ );
8      }
9    }
10 }
```

CS 225

#29: Heap Operations

April 22-24, 2026 V. Kindratenko

Q: How do we construct a heap given data?



-	B	U	I	L	D	H	E	A	P	N	O	W			

An O(n) approach to buildHeap:

Heap.cpp (partial)

```
1  template <class T>
2  void Heap<T>::buildHeap() {
3    for (unsigned i = parent(size); i > 0; i--) {
4      heapifyDown(i);
5    }
6  }
```

Theorem: The running time of buildHeap on array of size n is: _____.

Strategy:

- We know that constant work is done based on the distance a node is away from the root (eg: it's height).
- Therefore, the running time is proportional to the sum of the heights of the heights of all the nodes.
- We will work towards creating a proof around the sum of the heights of all the nodes.

Define S(h):

Let **S(h)** denote the sum of the heights of all nodes in a complete tree of height **h**.

S(0) =

S(1) =

S(h) =

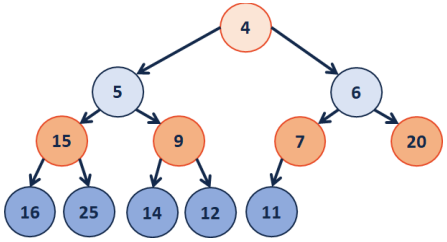
Proof of S(h) by Induction:

Base Case(s):

General Case:

Finally, finding the running time:

Heap Sort



Algorithm:

1.

2.

3.

Running time?

Why do we care about another sort?

CS 225

#30: Disjoint Sets

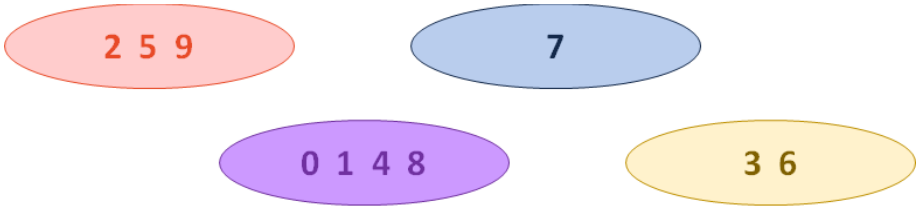
April 22-24, 2026 V. Kindratenko

Disjoint Sets

Let **R** be an equivalence relation on *us* where **(s, t) ∈ R** if **s** and **t** have the same favorite among:

{ ____, ____, _____, ____, _____, ____ }

Examples:

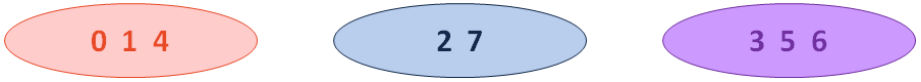


Building Disjoint Sets:

- Maintain a collection S = {s_0, s_1, … s_k}
- Each set has a representative member.
- ADT:

  void makeSet(const T & t);
  void union(const T & k1, const T & k2);
  T & find(const T & k);

Implementation #1:



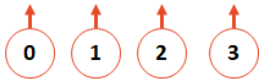
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]

Operation: find(k)

Operation: union(k1, k2)

Implementation #2:

- We will continue to use an array where the index is the key
- The value of the array is:
  - **-1**, if we have found the representative element
  - **The index of the parent**, if we haven't found the rep. element



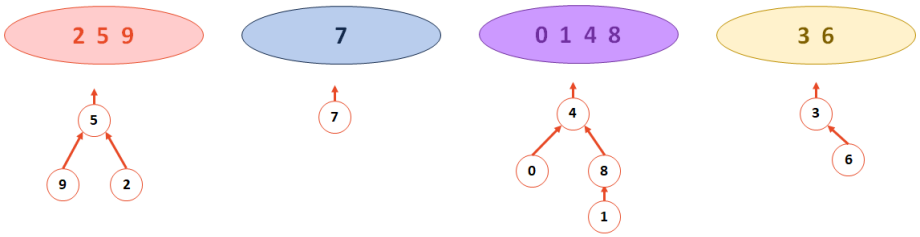
[0]	[1]	[2]	[3]

[0]	[1]	[2]	[3]

[0]	[1]	[2]	[3]

[0]	[1]	[2]	[3]

Example:



4	8	5	6	-1	-1	-1	-1	4	5
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]

...what is the error in this table?

Implementation

DisjointSets.cpp (partial)

```
1  int DisjointSets::find(int i) {
2    if ( s[i] < 0 ) { return i; }
3    else { return _find( s[i] ); }
4  }
```

What is the running time of `find`?

What is the ideal UpTree?

DisjointSets.cpp (partial)

```
1  void DisjointSets::union(int r1, int r2) {
2
3
4  }
```

How do we want to union the two UpTrees?

CS 225 – Things To Be Doing:

1. mp5 due next Monday (April 27)
2. lab_hash due this Sunday (April 26)
3. Quiz 5 this Thursday (April 23)